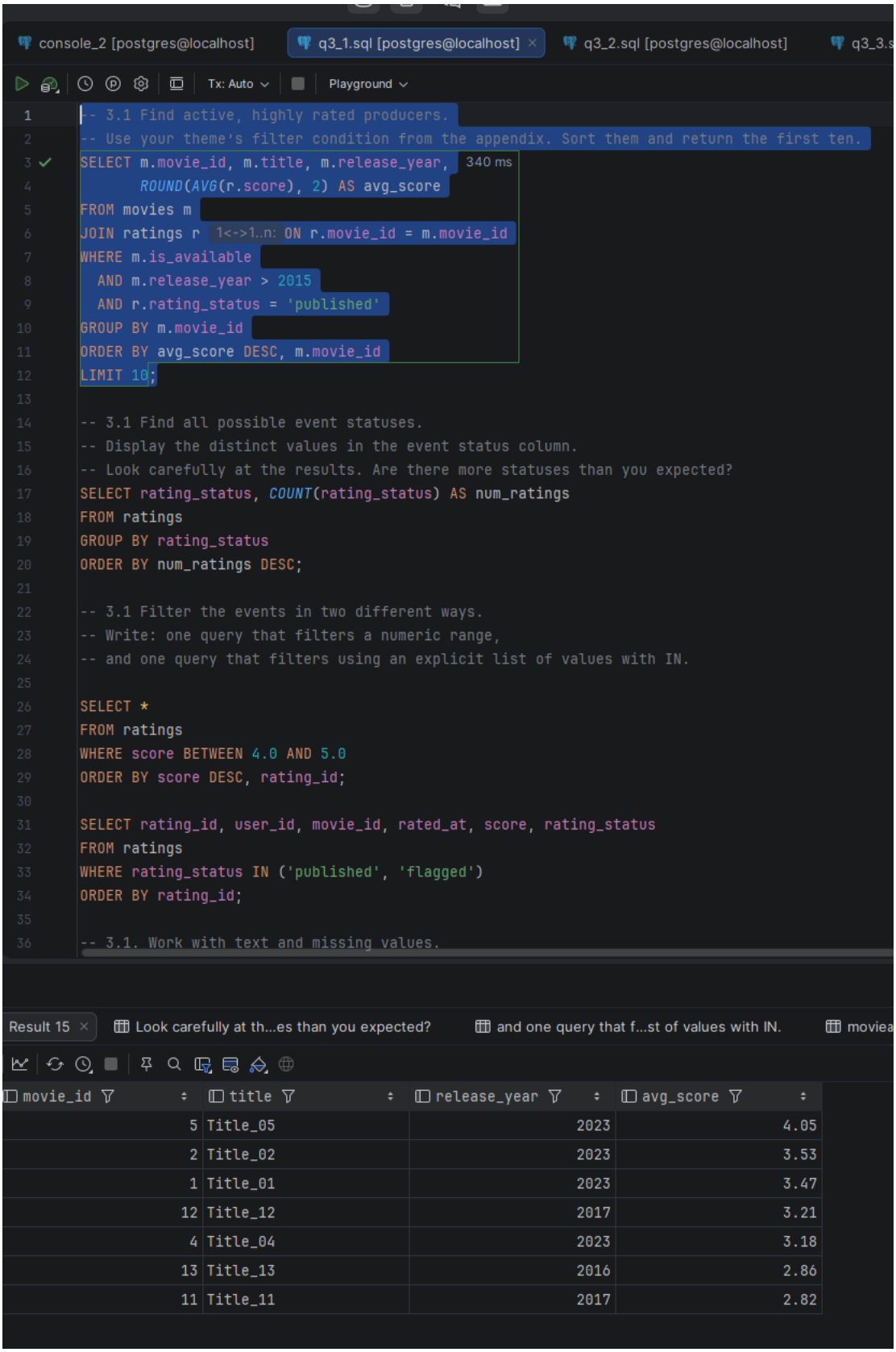
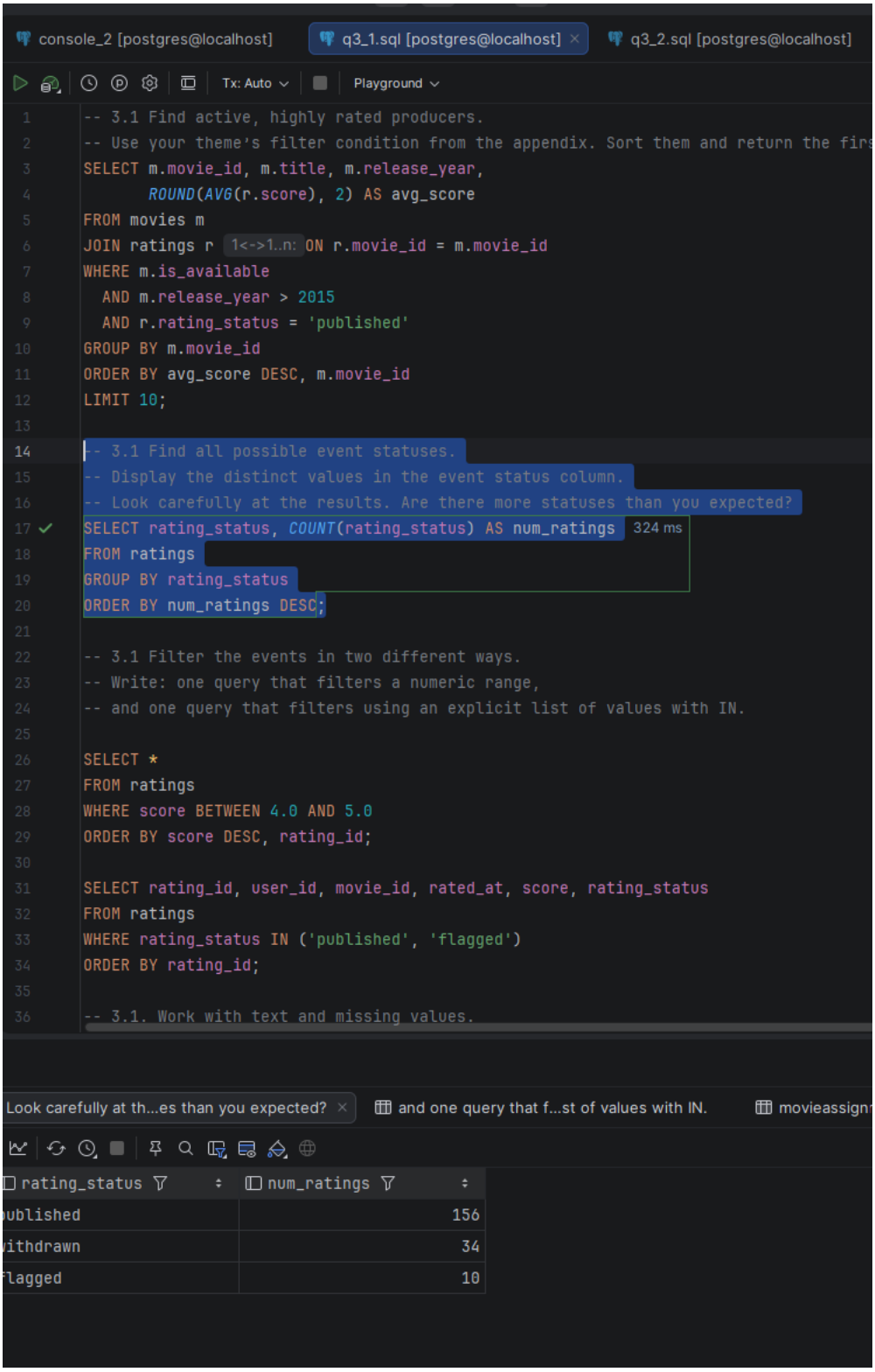
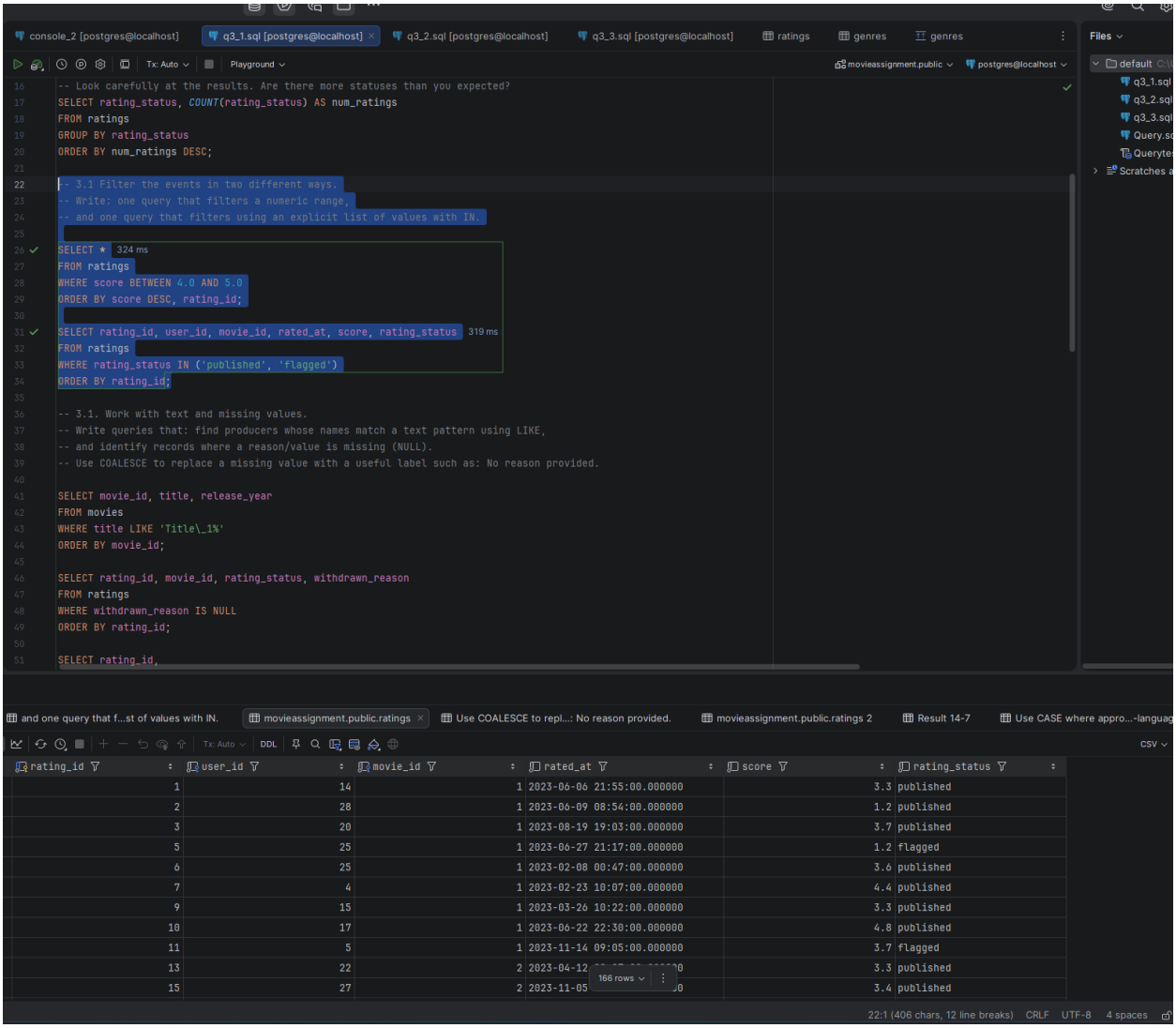
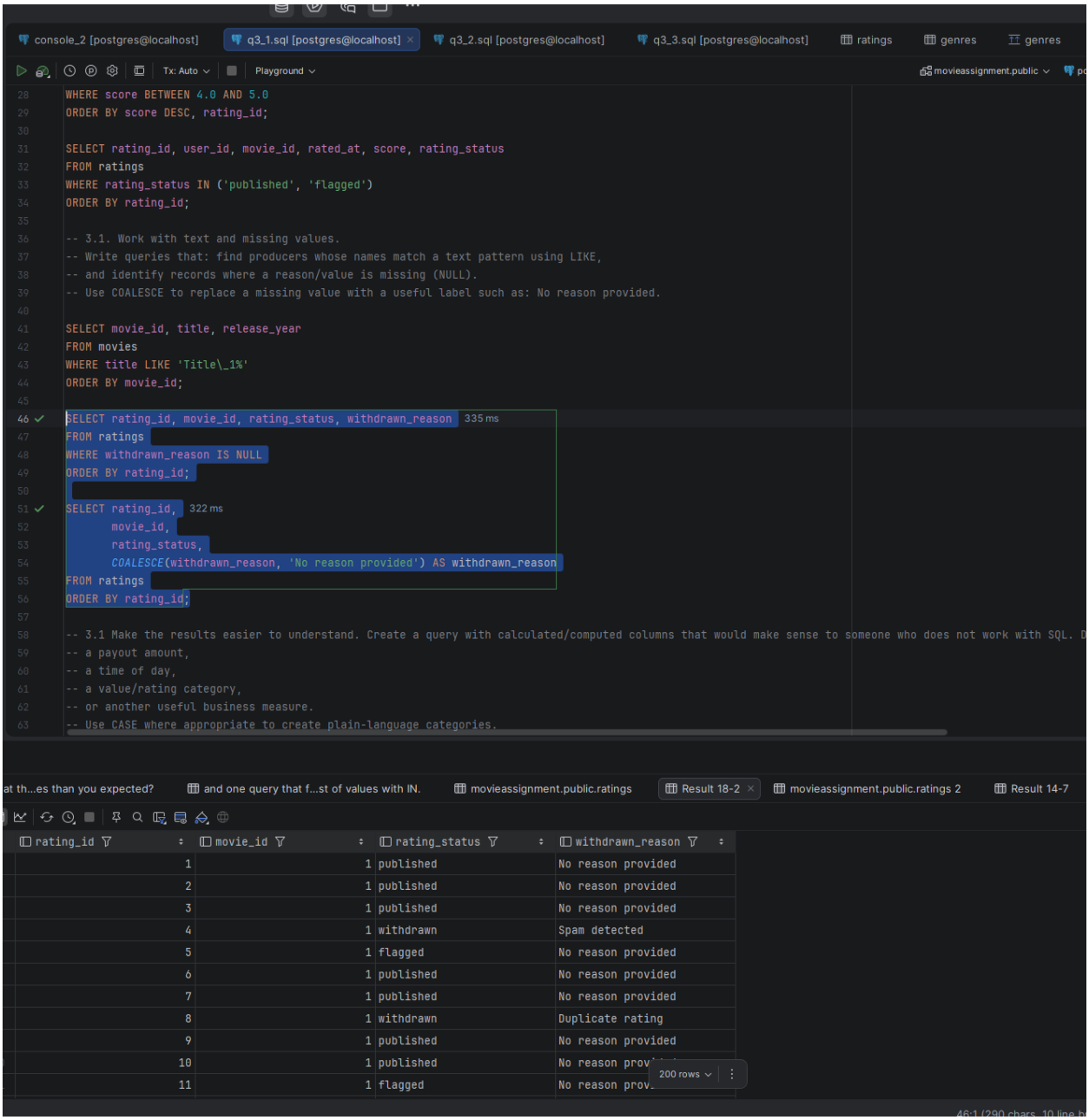
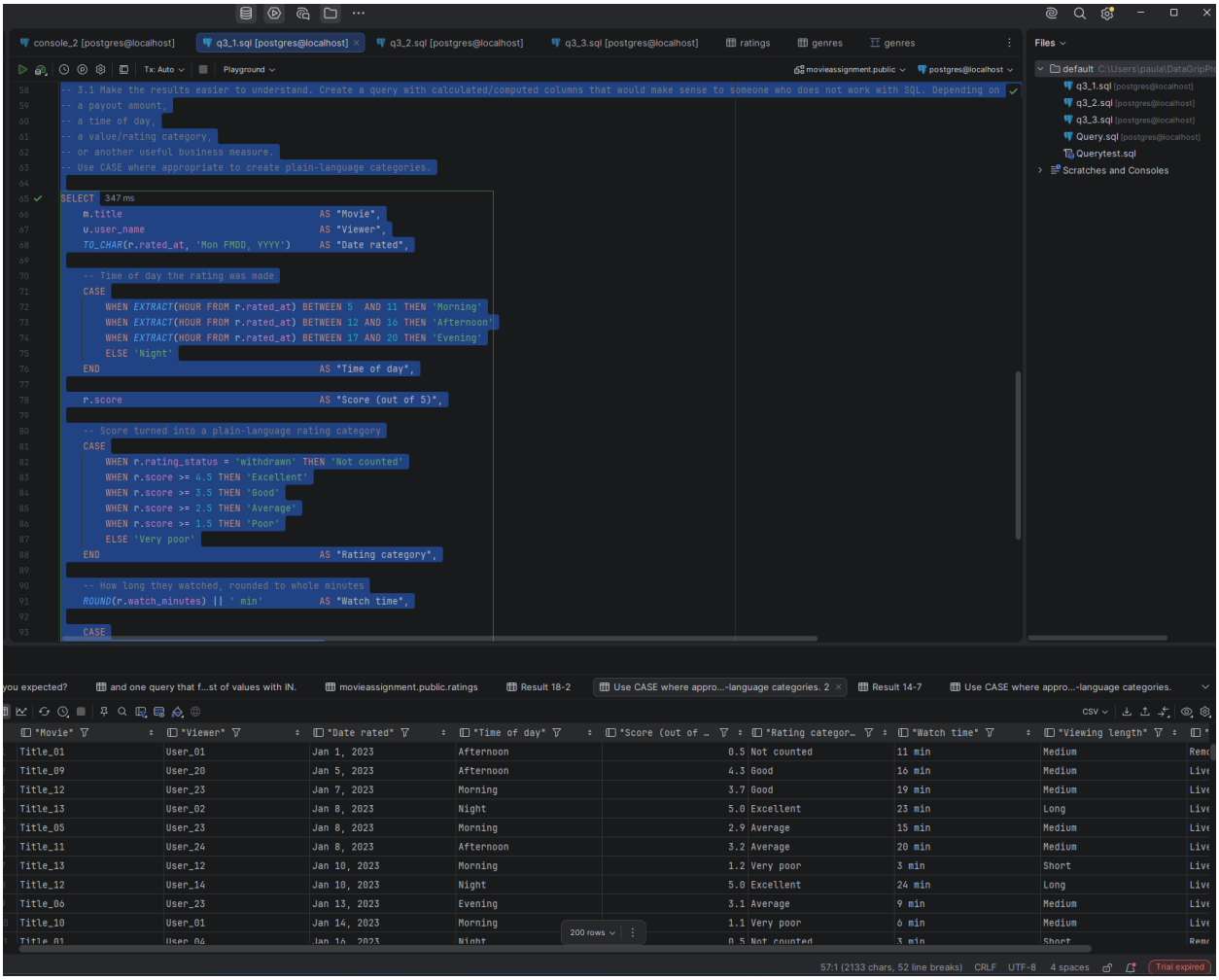

console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
Tx: Auto
Playground
-- 3.1 Find active, highly rated producers.
-- Use your theme's filter condition from the appendix. Sort them and return the first ten.
SELECT m.movie_id, m.title, m.release_year, 340 ms
ROUND(AVG(r.score), 2) AS avg_score
FROM movies m
JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id
WHERE m.is_available
AND m.release_year > 2015
AND r.rating_status = 'published'
GROUP BY m.movie_id
ORDER BY avg_score DESC, m.movie_id
LIMIT 10;
-- 3.1 Find all possible event statuses.
-- Display the distinct values in the event status column.
-- Look carefully at the results. Are there more statuses than you expected?
SELECT rating_status, COUNT(rating_status) AS num_ratings
FROM ratings
GROUP BY rating_status
ORDER BY num_ratings DESC;
-- 3.1 Filter the events in two different ways.
-- Write: one query that filters a numeric range,
-- and one query that filters using an explicit list of values with IN.
SELECT *
FROM ratings
WHERE score BETWEEN 4.0 AND 5.0
ORDER BY score DESC, rating_id;
SELECT rating_id, user_id, movie_id, rated_at, score, rating_status
FROM ratings
WHERE rating_status IN ('published', 'flagged')
ORDER BY rating_id;
-- 3.1. Work with text and missing values.
Result 15
Look carefully at th...es than you expected?
and one query that f...st of values with IN.
movie_id
title
release_year
avg_score
5 Title_05 2023 4.05
2 Title_02 2023 3.53
1 Title_01 2023 3.47
12 Title_12 2017 3.21
4 Title_04 2023 3.18
13 Title_13 2016 2.86
11 Title_11 2017 2.82


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
Tx: Auto
Playground
-- 3.1 Find active, highly rated producers.
-- Use your theme's filter condition from the appendix. Sort them and return the firs
SELECT m.movie_id, m.title, m.release_year,
ROUND(AVG(r.score), 2) AS avg_score
FROM movies m
JOIN ratings r 1<->1..n: ON r.movie_id = m.movie_id
WHERE m.is_available
AND m.release_year > 2015
AND r.rating_status = 'published'
GROUP BY m.movie_id
ORDER BY avg_score DESC, m.movie_id
LIMIT 10;
-- 3.1 Find all possible event statuses.
-- Display the distinct values in the event status column.
-- Look carefully at the results. Are there more statuses than you expected?
SELECT rating_status, COUNT(rating_status) AS num_ratings
324 ms
FROM ratings
GROUP BY rating_status
ORDER BY num_ratings DESC;
-- 3.1 Filter the events in two different ways.
-- Write: one query that filters a numeric range,
-- and one query that filters using an explicit list of values with IN.
SELECT *
FROM ratings
WHERE score BETWEEN 4.0 AND 5.0
ORDER BY score DESC, rating_id;
SELECT rating_id, user_id, movie_id, rated_at, score, rating_status
FROM ratings
WHERE rating_status IN ('published', 'flagged')
ORDER BY rating_id;
-- 3.1. Work with text and missing values.
Look carefully at th...es than you expected?
and one query that f...st of values with IN.
movieassign
rating_status
num_ratings
ublished
156
ithdrawn
34
lagged
10


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
genres
Files
Tx: Auto
Playground
movieassignment.public
postgres@localhost
-- Look carefully at the results. Are there more statuses than you expected?
SELECT rating_status, COUNT(rating_status) AS num_ratings
FROM ratings
GROUP BY rating_status
ORDER BY num_ratings DESC;
-- 3.1 Filter the events in two different ways.
-- Write: one query that filters a numeric range,
-- and one query that filters using an explicit list of values with IN.
SELECT * 324 ms
FROM ratings
WHERE score BETWEEN 4.0 AND 5.0
ORDER BY score DESC, rating_id;
SELECT rating_id, user_id, movie_id, rated_at, score, rating_status 319 ms
FROM ratings
WHERE rating_status IN ('published', 'flagged')
ORDER BY rating_id;
-- 3.1. Work with text and missing values.
-- Write queries that: find producers whose names match a text pattern using LIKE,
-- and identify records where a reason/value is missing (NULL).
-- Use COALESCE to replace a missing value with a useful label such as: No reason provided.
SELECT movie_id, title, release_year
FROM movies
WHERE title LIKE 'Title_1%'
ORDER BY movie_id;
SELECT rating_id, movie_id, rating_status, withdrawn_reason
FROM ratings
WHERE withdrawn_reason IS NULL
ORDER BY rating_id;
SELECT rating_id,
and one query that f...st of values with IN.
movieassignment.public.ratings
Use COALESCE to repl...: No reason provided.
movieassignment.public.ratings 2
Result 14-7
Use CASE where appro...-languag
DDL
CSV
rating_id
user_id
movie_id
rated_at
score
rating_status
1 14 1 2023-06-06 21:55:00.000000 3.3 published
2 28 1 2023-06-09 08:54:00.000000 1.2 published
3 20 1 2023-08-19 19:03:00.000000 3.7 published
5 25 1 2023-06-27 21:17:00.000000 1.2 flagged
6 25 1 2023-02-08 00:47:00.000000 3.6 published
7 4 1 2023-02-23 10:07:00.000000 4.4 published
9 15 1 2023-03-26 10:22:00.000000 3.3 published
10 17 1 2023-06-22 22:30:00.000000 4.8 published
11 5 1 2023-11-14 09:05:00.000000 3.7 flagged
13 22 2 2023-04-12 3.3 published
15 27 2 2023-11-05 3.4 published
166 rows
22:1 (406 chars, 12 line breaks)
CRLF
UTF-8
4 spaces


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
genres
Tx: Auto
Playground
movieassignment.public
WHERE score BETWEEN 4.0 AND 5.0
ORDER BY score DESC, rating_id;
SELECT rating_id, user_id, movie_id, rated_at, score, rating_status
FROM ratings
WHERE rating_status IN ('published', 'flagged')
ORDER BY rating_id;
-- 3.1. Work with text and missing values.
-- Write queries that: find producers whose names match a text pattern using LIKE,
-- and identify records where a reason/value is missing (NULL).
-- Use COALESCE to replace a missing value with a useful label such as: No reason provided.
SELECT movie_id, title, release_year
FROM movies
WHERE title LIKE 'Title_1%'
ORDER BY movie_id;
SELECT rating_id, movie_id, rating_status, withdrawn_reason
335 ms
FROM ratings
WHERE withdrawn_reason IS NULL
ORDER BY rating_id;
SELECT rating_id,
322 ms
movie_id,
rating_status,
COALESCE(withdrawn_reason, 'No reason provided') AS withdrawn_reason
FROM ratings
ORDER BY rating_id;
-- 3.1 Make the results easier to understand. Create a query with calculated/computed columns that would make sense to someone who does not work with SQL.
-- a payout amount,
-- a time of day,
-- a value/rating category,
-- or another useful business measure.
-- Use CASE where appropriate to create plain-language categories.
at th...es than you expected?
and one query that f...st of values with IN.
movieassignment.public.ratings
Result 18-2
movieassignment.public.ratings 2
Result 14-7
rating_id
movie_id
rating_status
withdrawn_reason
1 1 published No reason provided
2 1 published No reason provided
3 1 published No reason provided
4 1 withdrawn Spam detected
5 1 flagged No reason provided
6 1 published No reason provided
7 1 published No reason provided
8 1 withdrawn Duplicate rating
9 1 published No reason provided
10 1 published No reason prov
11 1 flagged No reason prov
200 rows


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
genres
Files
default C:\Users\paula\DataGripPr
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
Query.sql [postgres@localhost]
Querytest.sql
Scratches and Consoles
Tx: Auto
Playground
movieassignment.public
postgres@localhost
-- 3.1 Make the results easier to understand. Create a query with calculated/computed columns that would make sense to someone who does not work with SQL. Depending on
-- a payout amount,
-- a time of day,
-- a value/rating category,
-- or another useful business measure.
-- Use CASE where appropriate to create plain-language categories.
SELECT 347 ms
m.title AS "Movie",
u.user_name AS "Viewer",
TO_CHAR(r.rated_at, 'Mon FMDD, YYYY') AS "Date rated",
-- Time of day the rating was made
CASE
WHEN EXTRACT(HOUR FROM r.rated_at) BETWEEN 5 AND 11 THEN 'Morning'
WHEN EXTRACT(HOUR FROM r.rated_at) BETWEEN 12 AND 16 THEN 'Afternoon'
WHEN EXTRACT(HOUR FROM r.rated_at) BETWEEN 17 AND 20 THEN 'Evening'
ELSE 'Night'
END AS "Time of day",
r.score AS "Score (out of 5)",
-- Score turned into a plain-language rating category
CASE
WHEN r.rating_status = 'withdrawn' THEN 'Not counted'
WHEN r.score >= 4.5 THEN 'Excellent'
WHEN r.score >= 3.5 THEN 'Good'
WHEN r.score >= 2.5 THEN 'Average'
WHEN r.score >= 1.5 THEN 'Poor'
ELSE 'Very poor'
END AS "Rating category",
-- How long they watched, rounded to whole minutes
ROUND(r.watch_minutes) || ' min' AS "Watch time",
CASE
you expected?
and one query that f...st of values with IN.
movieassignment.public.ratings
Result 18-2
Use CASE where appro...-language categories. 2
Result 14-7
Use CASE where appro...-language categories.
CSV
"Movie"
"Viewer"
"Date rated"
"Time of day"
"Score (out of …
"Rating categor…
"Watch time"
"Viewing length"
Title_01 User_01 Jan 1, 2023 Afternoon 0.5 Not counted 11 min Medium
Title_09 User_20 Jan 5, 2023 Afternoon 4.3 Good 16 min Medium
Title_12 User_23 Jan 7, 2023 Morning 3.7 Good 19 min Medium
Title_13 User_02 Jan 8, 2023 Night 5.0 Excellent 23 min Long
Title_05 User_23 Jan 8, 2023 Morning 2.9 Average 15 min Medium
Title_11 User_24 Jan 8, 2023 Afternoon 3.2 Average 20 min Medium
Title_13 User_12 Jan 10, 2023 Morning 1.2 Very poor 3 min Short
Title_12 User_14 Jan 10, 2023 Night 5.0 Excellent 24 min Long
Title_06 User_23 Jan 13, 2023 Evening 3.1 Average 9 min Medium
Title_10 User_01 Jan 14, 2023 Morning 1.1 Very poor 6 min Medium
200 rows
57:1 (2133 chars, 52 line breaks) CRLF UTF-8 4 spaces
Trial expired